End-to-end hands-on RAG implementation

(Triển khai RAG thực tế từ đầu đến cuối)

Tài liệu này sẽ hướng dẫn bạn triển khai mô kiến trúc RAG hoàn chỉnh một các dễ hiểu nhất thông qua một vài khái niệm quan trong và các hướng dẫn code thực tế.

1. Giới thiệu

Tài liệu này hướng dẫn bạn triển khai một hệ thống **Retrieval-Augmented Generation (RAG)** hoàn chỉnh theo cách **hands-on step-by-step**.

Mục tiêu của giáo trình:

- Hiểu **bản chất kiến trúc RAG**
- Tự triển khai **chatbot truy vấn tài liệu**
- Hiểu **flow hoạt động thực tế của hệ thống AI**
- Xây dựng nền tảng để mở rộng lên **production RAG system**

Trong tài liệu này bạn sẽ:

- build **vector database từ tài liệu PDF**
- tạo **chatbot hỏi đáp tài liệu**
- hiểu **2 pipeline quan trọng trong RAG**

2. Định nghĩa RAG

Retrieval-Augmented Generation (RAG) là kiến trúc kết hợp:

1. **Information Retrieval**
2. **Large Language Model Generation**

Thay vì để LLM trả lời dựa hoàn toàn vào dữ liệu đã train, RAG sẽ:

1. Tìm kiếm thông tin liên quan trong dữ liệu riêng
2. Đưa thông tin đó vào **prompt context**
3. Yêu cầu LLM trả lời dựa trên context

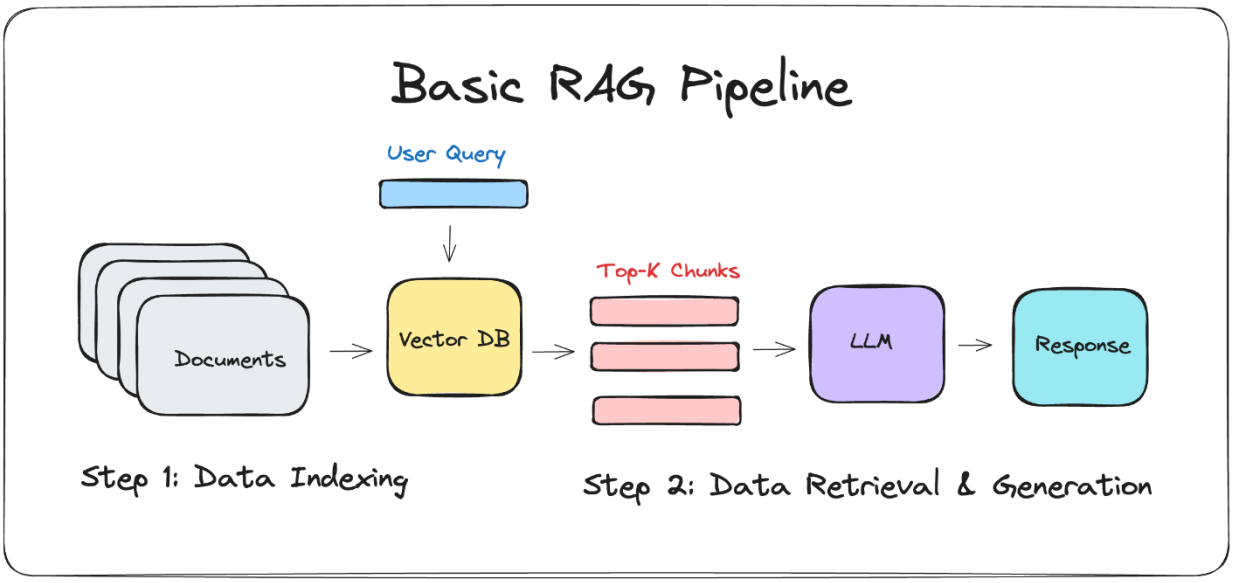
Lợi ích

- Cập nhật kiến thức **real-time**
- Giảm **hallucination**
- Kiểm soát **nguồn dữ liệu**

3. Tổng quan kiến trúc RAG

General RAG Pipeline

Pipeline tổng quát:



Ba thành phần chính của hệ thống RAG:

- **Embedding Model**
- **Vector Database**
- **Large Language Model (LLM)**

Trong hệ thống thực tế, RAG luôn tách thành **2 pipeline riêng biệt**, gồm:

- Phần Ingestion (Nap data) chạy **offline**
- Phần Query chạy **real-time**

3.1 Data Ingestion Pipeline (Offline)

Pipeline này dùng để **xây dựng knowledge base**.

Documents → Loader → Text Cleaning → Chunking → Embedding → Vector Database → Save Index

Pipeline này thường chạy:

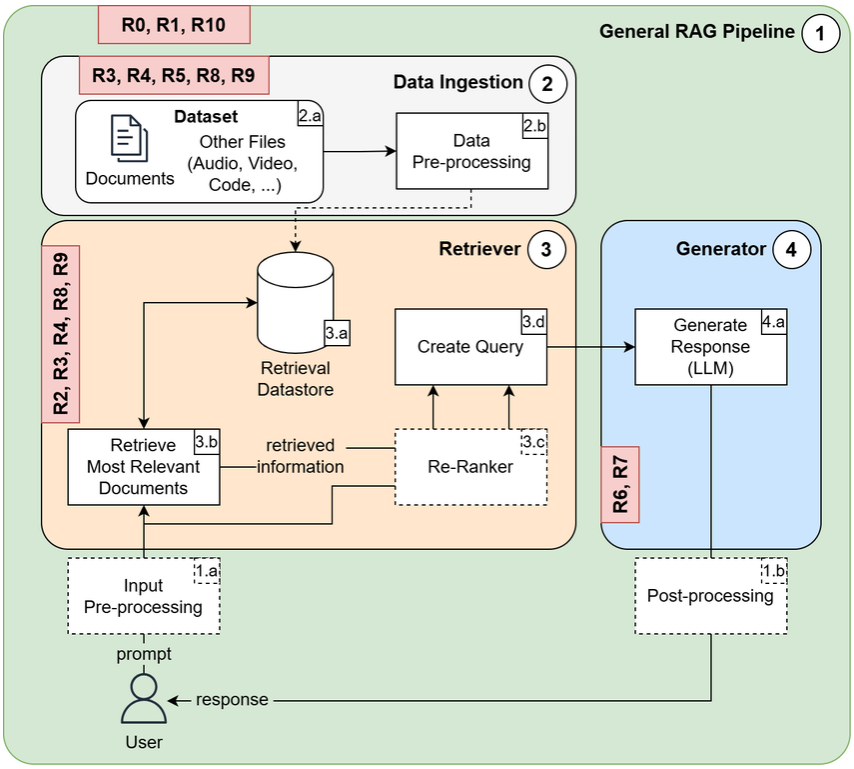
- Khi **import dữ liệu mới**
- Khi **rebuild knowledge base**

3.2 Query Pipeline (Online)

Pipeline này dùng khi **user đặt câu hỏi**.

User Query → Embedding Query → Vector Search → Retrieve Top-K Chunks → Prompt LLM → Answer

4. Ví dụ pipeline RAG thực tế



Trong production pipeline thường có thêm:

- Metadata filtering
- Reranking
- Context compression

5. Giới thiệu các thư viện sẽ dùng trong project:

5.1. google-genai

Thư viện dùng để truy cập các mô hình AI của **Google Gemini**.

Trong hệ thống RAG, thư viện này được dùng để:

- tạo **embedding** cho văn bản
- tạo **embedding cho câu hỏi**
- dùng **LLM để sinh câu trả lời cuối cùng**

Ví dụ pipeline:

Text → Gemini Embedding → Vector

5.2. SentenceTransformers

Đây là thư viện Python dùng để **chuyển câu hoặc đoạn văn thành vector embedding**.
Thư viện được xây dựng trên **PyTorch** và sử dụng các model từ **Hugging Face**.

Ví dụ code:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

embedding = model.encode("Artificial Intelligence is amazing")
```

Kết quả:

```
[0.21, -0.33, 0.11, 0.78, ...]
```

Vector này có **384 chiều**.

Model thường dùng là **all-MiniLM-L6-v2**.

Đặc điểm:

Thuộc tính	Giá trị
embedding dimension	384
kích thước model	~80MB
tốc độ	nhanh
ứng dụng	semantic search, RAG

Model được train để:

```
hai câu có nghĩa giống nhau → vector gần nhau
hai câu khác nghĩa → vector xa nhau
```

Ví dụ:

```
"What is AI?"
"What is artificial intelligence?"
```

→ vector sẽ rất gần nhau trong **semantic space**.

Embedding tạo ra từ model này thường được lưu vào **FAISS** để tìm kiếm.

5.3. FAISS

FAISS là thư viện dùng để **tìm kiếm vector giống nhau (vector similarity search)** với tốc độ rất cao.
Thư viện này được phát triển bởi **Meta**.

Trong hệ thống **RAG**, FAISS thường đóng vai trò như một **vector database đơn giản**.

FAISS lưu dữ liệu như thế nào

FAISS **không lưu văn bản**, nó chỉ lưu **vector embedding**.

Ví dụ:

```
Chunk text:
"Artificial Intelligence is transforming industry"
```

Sau khi embedding:

```
[0.12, -0.44, 0.89, 0.32, ...]
```

FAISS sẽ lưu dạng:

```
vector_1
vector_2
vector_3
vector_4
```

Ví dụ một index đơn giản:

```
IndexFlatL2

[vector_1]
[vector_2]
[vector_3]
```

Khi người dùng đặt câu hỏi
Pipeline:
User Question → Embedding → Query Vector → FAISS Search → Top-K Similar Vectors

Ví dụ kết quả:

```
index: [3, 10, 25]
distance: [0.12, 0.18, 0.20]
```

Nghĩa là:

```
vector_3
vector_10
vector_25
```

là các đoạn văn **giống nhất với câu hỏi**.

Một số lưu ý về cách hoạt động của FAISS:

FAISS không lưu metadata
FAISS chỉ lưu **vector**, không lưu:

- text
- title
- page
- document_id

Vì vậy **cần tự lưu mapping.**

Ví dụ:

```
chunks = [
 "AI is a field of computer science...",
 "Machine learning is a subset of AI...",
 "Deep learning uses neural networks..."
]
```

FAISS lưu:

```
vector_0
vector_1
vector_2
```

Nếu kết quả search trả:

```
index = 1
```

thì ta lấy lại text:

```
chunks[1]
```

Các vector database hiện đại

FAISS chỉ là **library**. Các vector database hiện đại thường có thêm:

Feature	FAISS	Vector DB hiện đại
vector search	✓	✓
metadata	✗	✓
filter	✗	✓
REST API	✗	✓
GUI	✗	✓
distributed	✗	✓

Ví dụ:

- **Chroma**
- **Milvus**
- **Qdrant**
- **Weaviate**

Các hệ này lưu dữ liệu dạng:

- Vector
- Metadata
- Document

Ví dụ record:

```
{
 vector: [0.12, 0.44, ...],
 text: "AI is transforming the world",
```

```
 metadata: {
     source: "ai_book.pdf",
     page: 12
 }
}
```

Nếu có **GUI**, bạn có thể thấy:

```
Collection: documents

ID: 23
Text: "Artificial Intelligence is transforming industries..."
Embedding: [0.23, -0.11, ...]
Metadata:
   source: ai_book.pdf
   page: 14
```

Có thể:

- search
- filter
- xem document

Vì sao FAISS thường dùng trong RAG demo

✓ rất nhanh
✓ dễ cài đặt
✓ chạy local
✓ không cần server

Nhưng trong hệ production thường dùng:

- Milvus
- Weaviate
- Qdrant
- Pinecone

5.4. NumPy

Thư viện tính toán số học trong Python, dùng để làm việc với vector và ma trận.

Trong RAG, NumPy thường dùng để:

- Lưu embedding
- Tính toán vector
- Chuyển dữ liệu cho FAISS

5.5. pypdf

Thư viện dùng để đọc và trích xuất nội dung từ file PDF.

Pipeline thường sử dụng:

PDF → pypdf đọc file → trích xuất text → chunking

5.6. python-dotenv

Thư viện dùng để đọc API key từ file .env và bảo mật thông tin cấu hình.

6. Phần thực hành

6.1. Chuẩn bị môi trường

Cài Python:

Cài **Python 3.10+**

IDE đề xuất:

- VSCode
- PyCharm

Tạo Virtual Environment

Mac / Linux

```
python -m venv rag_env
source rag_env/bin/activate
```

Windows

```
python -m venv rag_env
rag_env\Scripts\activate
```

Cài thư viện

```
pip install google-genai
pip install sentence-transformers
pip install faiss-cpu
pip install numpy
pip install pypdf
pip install python-dotenv
```

6.2 Cấu trúc Project

```
rag_project/

data/
   data_sample.pdf

pipeline/
   __init__.py
   build_index.py

query/
   __init__.py
   query_rag.py

rag/
   __init__.py
   loader.py
   chunker.py
   embedder.py
   retriever.py
   rag_chatbot.py
```

6.3 Tổng quan pipeline RAG trong project

Hệ thống gồm 2 pipeline chính:

1. Data Ingestion Pipeline:

Dùng để **xây dựng vector database từ tài liệu**

Flow:

PDF → Load → Chunk → Embedding → Vector DB → Save FAISS Index

2. Query Pipeline:

Dùng để **trả lời câu hỏi của user**

Flow:

User Question → Query Embedding → Vector Search → Retrieve Chunks → LLM → Answer

PHẦN 1 — DATA INGESTION PIPELINE

6.4 Load tài liệu

File: `rag/loader.py`

```
from pypdf import PdfReader

def load_pdf(path):

    reader = PdfReader(path)

    text = ""

    for page in reader.pages:
        text += page.extract_text()

    return text
```

6.5 Chunking văn bản

File: `rag/chunker.py`

```
def chunk_text(text, chunk_size=500, overlap=100):

    chunks = []

    start = 0

    while start < len(text):

        end = start + chunk_size

        chunk = text[start:end]

        chunks.append(chunk)

        start += chunk_size - overlap

    return chunks
```

Chunking giúp:

- Tăng **retrieval accuracy**
- Giảm **context noise**

6.6 Embedding

File: `rag/embedder.py`

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

def embed_text(text):
    print("đang xử lí text-embedding")

    vector = model.encode(text)

    print("xong task embedding!")

    return np.array(vector).astype("float32")
```

6.7 Build Vector Database

File: `pipeline/build_index.py`

```
from rag.loader import load_pdf
from rag.chunker import chunk_text
from rag.embedder import embed_text

import faiss
import numpy as np
import pickle

text = load_pdf("data/data_sample.pdf")

chunks = chunk_text(text)

embeddings = []

for chunk in chunks:
    embeddings.append(embed_text(chunk))

dimension = len(embeddings[0])

index = faiss.IndexFlatL2(dimension)

index.add(np.array(embeddings).astype("float32"))

faiss.write_index(index, "vector.index")

with open("chunks.pkl", "wb") as f:
    pickle.dump(chunks, f)

print("Vector database built successfully")
```

Chạy Data Ingestion pipeline:

```
python -m pipeline.build_index
```

PHẦN 2 — QUERY PIPELINE

6.8 Retrieval

File: `rag/retriever.py`

```
import faiss
import pickle
import numpy as np
from rag.embedder import embed_text

index = faiss.read_index("vector.index")

with open("chunks.pkl", "rb") as f:
    chunks = pickle.load(f)

def retrieve(query, k=3):

    query_vector = embed_text(query).astype("float32")

    query_vector = np.array([query_vector])

    distances, indices = index.search(query_vector, k)

    results = []

    for i in indices[0]:
        results.append(chunks[i])

    return results
```

6.9 RAG Chatbot

Tạo file `.env`

```
GEMINI_API_KEY=your_api_key_here
```

File: `rag/rag_chatbot.py`

```
from rag.retriever import retrieve
from google import genai
import os
from dotenv import load_dotenv

load_dotenv()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)
```

```
def ask_rag(question):

    contexts = retrieve(question)

    context_text = "\n\n".join(contexts)

    prompt = f"""
Use the following context to answer the question.

Context:
{context_text}

Question:
{question}

Answer:
"""

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    return response.text
```

6.10 CLI Chatbot

File: `query/query_rag.py`

```
from rag.rag_chatbot import ask_rag

print("RAG Chatbot Ready")

while True:

    query = input("\nAsk: ")

    if query.lower() == "exit":
        break

    answer = ask_rag(query)

    print("\nAnswer:\n")
    print(answer)
```

Chạy chatbot

```
python -m query.query_rag
```

Bạn thử hỏi chatbot các câu hỏi ví dụ như:

“xin chào”,

“các giai đoạn giái dục cơ bản”

Trong quá trình thử và đối chiếu tài liệu thì bạn sẽ thấy AI nó tìm kiếm thông tin và phản hồi lại cho bạn theo tài liệu và prompt context mà bạn đưa vào cho LLM, những câu hỏi không có thì nó sẽ dựa theo khả năng ngôn ngữ suy luận của nó (cụ thể là gemini 2.5 flask) nó sẽ phản hồi tương ứng lại cho người dùng

Để thoát giao diện hỏi của chatbot thì bạn tắt terminal hoặc Ctrl + C

7. Các lỗi thường gặp

Chunk quá lớn
→ retrieval kém chính xác

Chunk quá nhỏ
→ mất context

Context quá dài
→ LLM bị nhiễu

Không normalize vector
→ cosine similarity sai

8. Gợi ý phát triển project

Sau khi hoàn thành bản cơ bản, bạn có thể nâng cấp:

Hybrid Retrieval

```
BM25 + Dense Embedding
```

Reranking

Dùng cross encoder để chọn chunk tốt nhất.

Multi Query Retrieval

LLM sinh nhiều query để tìm context tốt hơn.

Context Compression

Loại bỏ context không liên quan.

VectorBD

Sử dụng các nền tảng VectorBD như Qdrant, FAISS, Milvus, Weaviate, supabase, … .

Giao diện Web App (FastAPI / NextJS)

Vừa nạp data vừa hỏi RAG AI trên web

Research-level retrieval optimization

Nghiên cứu chuyên sâu tối ưu thuật toán

9. Đánh giá hệ thống RAG

Một số metric phổ biến:

Recall@k

Chunk đúng có nằm trong top-k không.

MRR

Mean reciprocal rank.

Faithfulness

LLM có trả lời đúng theo context không.

10. Kết luận

Bạn đã hoàn thành triển khai hệ thống **RAG end-to-end**:

- Load tài liệu
- Chunking
- Embedding
- Vector Search
- Retrieval
- LLM Generation
- Chatbot CLI

Từ đây bạn có thể mở rộng sang:

- Web App (FastAPI / NextJS)

- Production Vector DB
- Multi-agent RAG
- Research-level retrieval optimization